Enrollment :-240840116062

Practical 4 : Perform Exploratory Data Analysis (EDA) using descriptive statistics and identify trends in datasets.

Step - 1 Import Required Libraries

```
#Import the Pandas LIbrary
# Pandas is used
import pandas as pd
#import the matplotlib
import matplotlib.pyplot as plt
```

Close

step-2 create a data set

step-2 create a data set

```
# creating  a data set
data={

       "Month": [ "January", "February", "March", "April", "May", "June", "July", "August", "September", "October", "Novemb
       "Unit_Sold" :[120, 135, 150, 145, 160, 175, 180, 170, 190, 210,  230, 250],
       "Revenue" : [ 120000, 135000, 150000, 145000, 160000, 175000, 180000, 170000, 190000, 210000, 230000, 250000],
       "Profit" :[20000, 23000, 25000, 24000, 27000, 30000, 31000, 29000, 33000, 37000, 41000, 45000]

}
df = pd.DataFrame(data)
print(df)
```

```
        Month  Unit_Sold  Revenue  Profit
0     January        120   120000   20000
1    February        135   135000   23000
2       March        150   150000   25000
3       April        145   145000   24000
4         May        160   160000   27000
5        June        175   175000   30000
6        July        180   180000   31000
7      August        170   170000   29000
8   September        190   190000   33000
9     October        210   210000   37000
10   November        230   230000   41000
11   December        250   250000   45000
```

step: 3 Inspect the dataset

```
df.head()# display first five
```

	Month	Unit_Sold	Revenue	Profit
0	January	120	120000	20000
1	February	135	135000	23000
2	March	150	150000	25000
3	April	145	145000	24000
4	May	160	160000	27000

Next steps: Generate code with df | New interactive sheet

```
# display ythe info
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12 entries, 0 to 11
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Month      12 non-null     object
```

```
 1   Unit_Sold  12 non-null     int64
 2   Revenue    12 non-null     int64
 3   Profit     12 non-null     int64
dtypes: int64(3), object(1)
memory usage: 516.0+ bytes
```

Step: 4 Generate discriptive statistic

```
#
df.describe()
```

	Unit_Sold	Revenue	Profit
count	12.000000	12.000000	12.000000
mean	176.250000	176250.000000	30416.666667
std	38.737168	38737.167963	7525.210155
min	120.000000	120000.000000	20000.000000
25%	148.750000	148750.000000	24750.000000
50%	172.500000	172500.000000	29500.000000
75%	195.000000	195000.000000	34000.000000
max	250.000000	250000.000000	45000.000000

step 5 calculate mean

```
# calculate the avg numbers of uint for permonth

average_units = df['Unit_Sold'].mean()

print("Average units sold per month:", average_units)
```

```
Average units sold per month: 176.25
```

Step: 6 calculate median

```
# calcutae the middel value of the units_Sold column
median_units = df['Unit_Sold'].median()

print("Median units sold per month:", median_units)
```

```
Median units sold per month: 172.5
```

Step: 7 calculate max and min

```
#caluate ther max and mini
# find the highest no of units sold in month
maximum_units = df['Unit_Sold'].max()
minimum_units = df['Unit_Sold'].min()

print("Maximum units sold per month:", maximum_units)
print("Minimum units sold per month:", minimum_units)
```

```
Maximum units sold per month: 250
Minimum units sold per month: 120
```

Step: 8 calculate standard deviation

```
#calculate the standar devaition unit sold
std_units = df["Unit_Sold"].std()
print("Standard deviation of units sold per month:", std_units)
```

```
Standard deviation of units sold per month: 38.73716796331634
```

Step: 9 find the month with highest sales

```
# find the month with highest  sales
# find the index of the row  having the maximum revenue.
highest_sales_index = df["Revenue"].idxmax()
```

```
# retive the month correspondig to the index
highest_sales_month = df.loc[highest_sales_index, "Month"]
print("Month with highest sales:", highest_sales_month)
# retrive the maximum revenue
highest_revenue = df.loc[highest_sales_index, "Revenue"]
print("Highest revenue:", highest_revenue)
```

```
Month with highest sales: December
Highest revenue: 250000
```

Step: 10 find the month with lowest sales

```
#  find the ,month with lowest sales

lowest_sales_index = df["Revenue"].idxmin()
#retirve the month correspinding
lowest_sales_month = df.loc[lowest_sales_index, "Month"]
print("Month with lowest sales:", lowest_sales_month)
# lowest revenue
lowest_revenue = df.loc[lowest_sales_index, "Revenue"]
print("Lowest revenue:", lowest_revenue)
```

```
Month with lowest sales: January
Lowest revenue: 120000
```

Step : 11 Calculate the total revenue

```
# calculate the lowest revenue generated during this year
total_revenue = df["Revenue"].sum()
print("Total revenue:", total_revenue)
```

```
Total revenue: 2115000
```

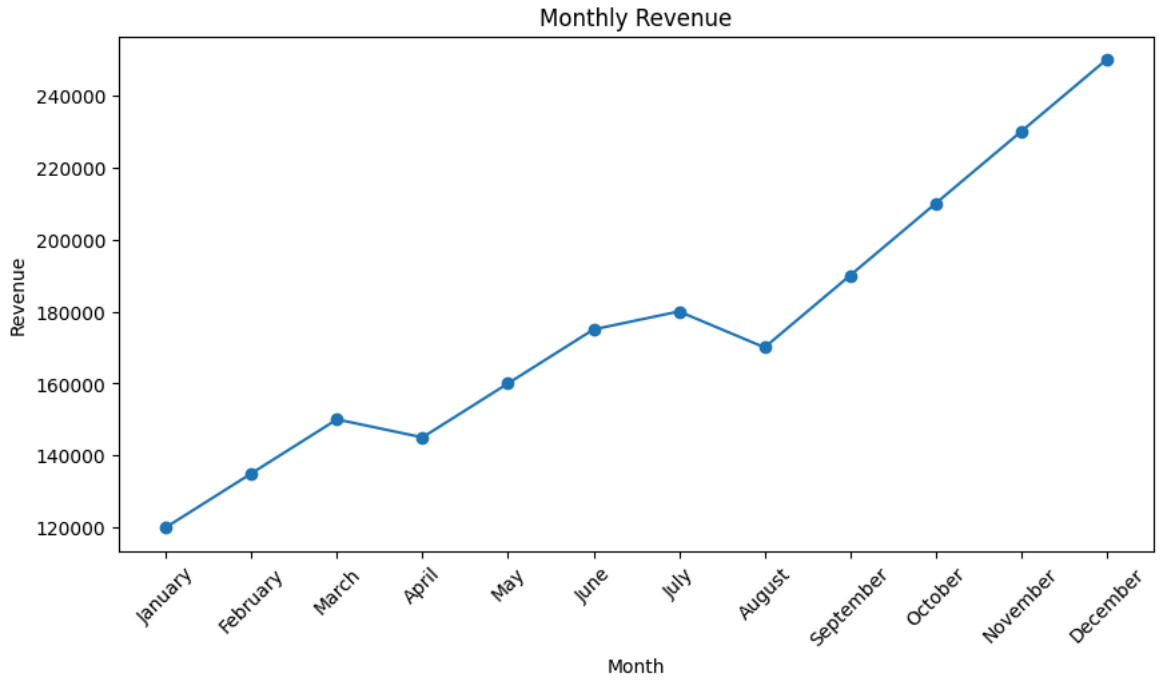
step : 12 calculate total profit

```
# calculate total profit
# Calculate the total profit generated during the year
total_profit = df["Profit"].sum()
print("Total profit:", total_profit)
```

```
Total profit: 365000
```

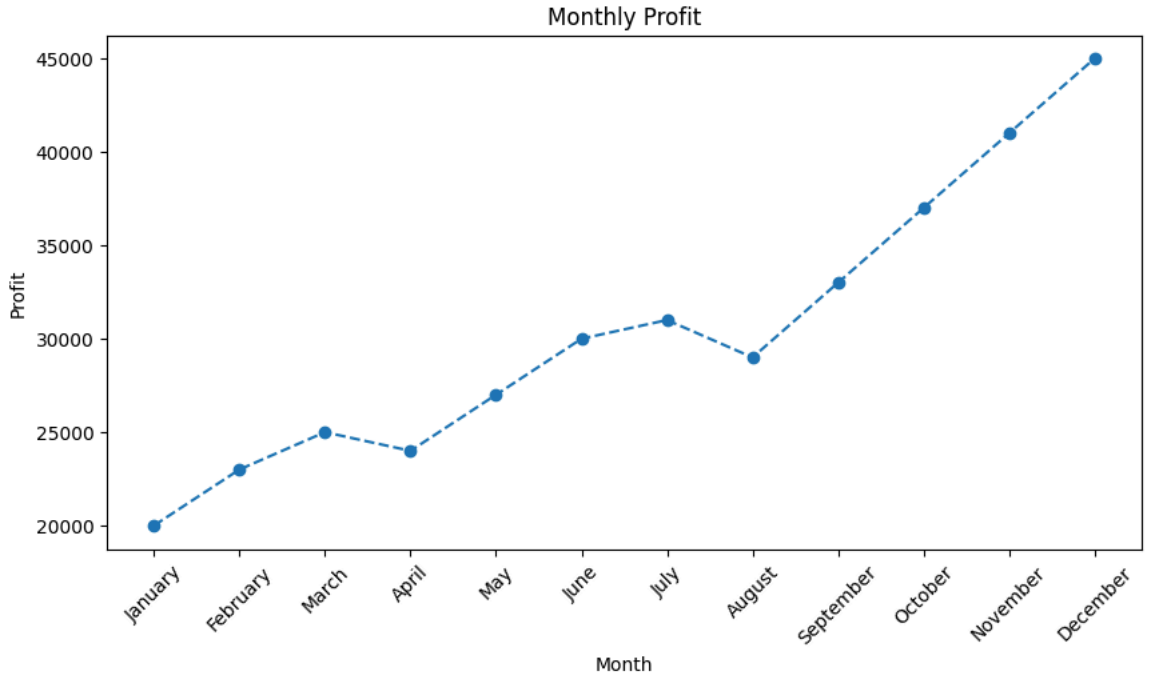
Step:13 identify the sales

```
# identify the sales Trend
# create a line chart  to visualize monthly revneue
plt.figure(figsize=(10, 5))
# ploting the revenue graph
plt.plot(df["Month"],df["Revenue"], marker='o', linestyle='-')
# LABELING X AXIS
plt.xlabel("Month")
# LABELING Y AXIS
plt.ylabel("Revenue")
# add the titel on the graph.
plt.title("Monthly Revenue")
# rotate the months name in the graph
plt.xticks(rotation=45)
# show the graph
plt.show()
```



Step:14 Analyze profit

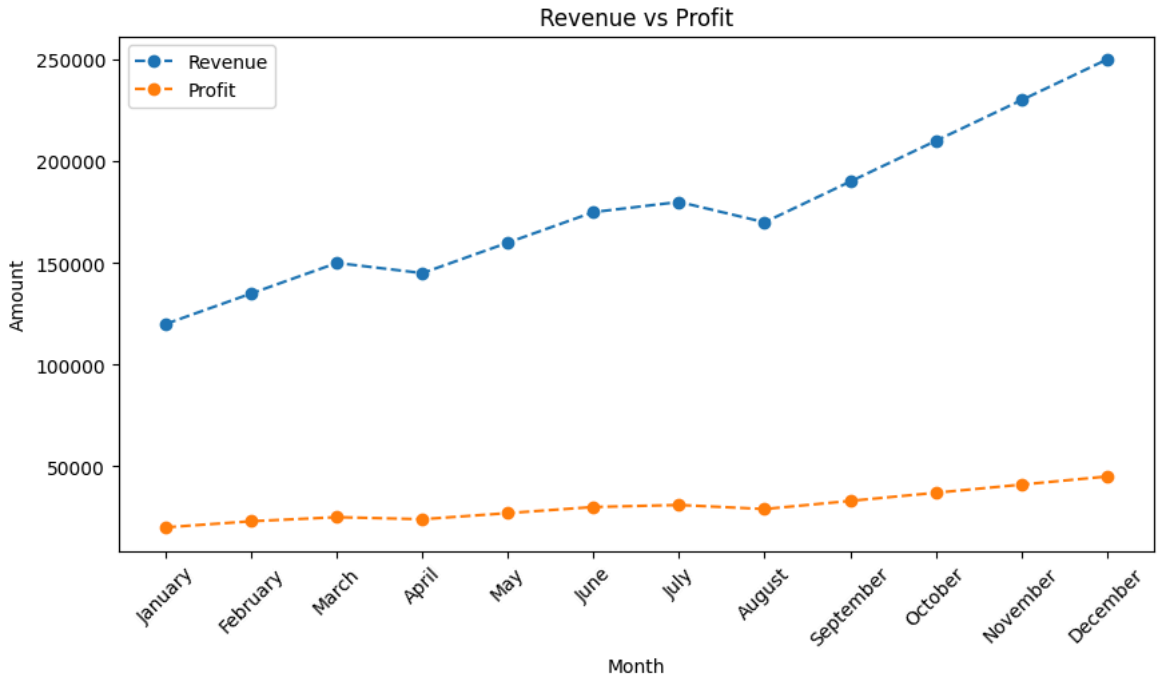
```
# create a line chart for monthly profit
plt.figure(figsize=(10, 5))
# PLOT TH GRAPH PROFIT
plt.plot(df["Month"],df["Profit"], marker='o', linestyle='--')
# LABEL FOR X AXIS
plt.xlabel("Month")
# LABEL FOR Y AXIS
plt.ylabel("Profit")
# TITLE FOR THE GRAPH
plt.title("Monthly Profit")
plt.xticks(rotation=45)
plt.show()
```



Step:15 compare revenue and profit both

```
#  comparee revenve and profit
plt.figure(figsize=(10, 5))
# plot revenue
plt.plot(df["Month"], df["Revenue"], marker='o', linestyle='--', label='Revenue')
# plot profit
```

```
plt.plot(df["Month"], df["Profit"], marker='o', linestyle='--', label='Profit')
# label for x - axis label
plt.xlabel("Month")
# label for y axis label
plt.ylabel("Amount")
# title for the graphs plot
plt.title("Revenue vs Profit")
# rotate the months naeme for better reading
plt.xticks(rotation=45)
# display the two lines
plt.legend()
# to print the graph
plt.show()
```



Step:16 Identify growth in sales

```
# calculate the percentage change in revenue
# from one month to the next
df["Revenue_Growth_%"] = df["Revenue"].pct_change()*100
# display the data set
df
```

	Month	Unit_Sold	Revenue	Profit	Revenue_Growth_%
0	January	120	120000	20000	NaN
1	February	135	135000	23000	12.500000
2	March	150	150000	25000	11.111111
3	April	145	145000	24000	-3.333333
4	May	160	160000	27000	10.344828
5	June	175	175000	30000	9.375000
6	July	180	180000	31000	2.857143
7	August	170	170000	29000	-5.555556
8	September	190	190000	33000	11.764706
9	October	210	210000	37000	10.526316
10	November	230	230000	41000	9.523810
11	December	250	250000	45000	8.695652

Next steps: Generate code with df | New interactive sheet

Step:17 Identify the months with increasing revenue

```
# identify the months with increasing Revenue
# Filter the dataset to display only those monthns
# where the growth is postive
increasing_revenue_months = df[df["Revenue_Growth_%"] > 0]
# display the months
increasing_revenue_months
```

	Month	Unit_Sold	Revenue	Profit	Revenue_Growth_%
1	February	135	135000	23000	12.500000
2	March	150	150000	25000	11.111111
4	May	160	160000	27000	10.344828
5	June	175	175000	30000	9.375000
6	July	180	180000	31000	2.857143
8	September	190	190000	33000	11.764706
9	October	210	210000	37000	10.526316
10	November	230	230000	41000	9.523810
11	December	250	250000	45000	8.695652

Next steps: Generate code with `increasing_revenue_months` New interactive sheet

Step:18 save the eda results

```
#  save the eda data set containg the revenue growth as a csv file
df.to_csv("EDA_Sales_analysis.csv",index=False)
print("update dataset saved to EDA_Sales_Analysis.csv")
```

```
update dataset saved to EDA_Sales_Analysis.csv
```

Step:19 Download the csv in google cola

```
# download the updated csv files
from google.colab import files
files.download("EDA_Sales_analysis.csv")
```